Protocolo de control TeleCon

Contrato de flujo de bytes con la aplicación Android

Referencia de firmware — cualquier MCU que implemente este contrato

Versión 1.2 2026

Resumen

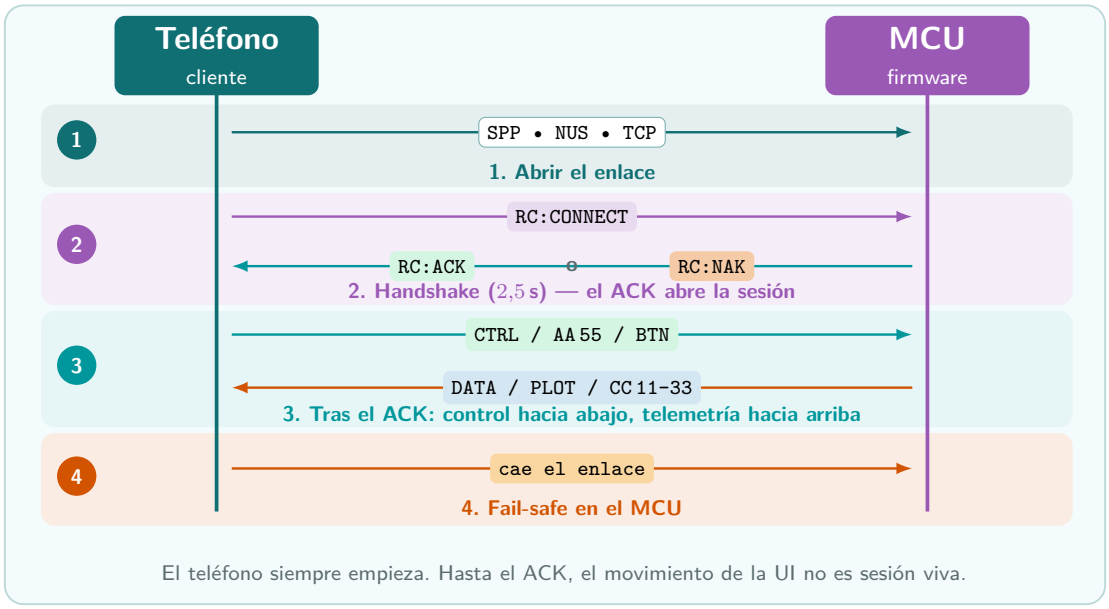
Este documento especifica cómo el firmware debe intercambiar bytes con TeleCon4ESP32 por Bluetooth Classic SPP, BLE Nordic UART o TCP. La aplicación es agnóstica al transporte: no exige un SoC Espressif, un nombre GAP concreto ni comandos AT de fabricante. Los sketches oficiales de Arduino son implementaciones de referencia de este contrato, no parte de la definición en el cable.

Control Panel y RC Vehicle Pro comparten el prefijo de aplicación `RC`. El vídeo HTTP de cámara (`/stream`, `/capture`) es una sesión aparte y queda fuera de alcance.

1. Arquitectura

El teléfono es siempre el cliente. Tras establecer el enlace subyacente, abre una sesión de aplicación. En el dispositivo hay que implementar tres capas:

1. **Transporte** — un flujo de octetos bidireccional suficientemente fiable (sección 2).
2. **Handshake de sesión** — una línea de texto `CONNECT` y una respuesta `ACK`/`NAK` (sección 4). El tiempo de espera en el teléfono es 2,5 s. Hasta el ACK, el control de la UI no es una sesión TeleCon viva, aunque el SO marque el socket como conectado.
3. **Carga** — líneas de texto Simple (sección 7) o tramas Binary (sección 8), según el ajuste del usuario y el campo `proto` de `CONNECT`.



Flujo teléfono–MCU. Verde = aceptar, naranja = rechazar, verde azulado = control, azul = telemetría.

El nombre anunciado por Bluetooth (por ejemplo `ESP32-TC-RC-BT-Simple` en el firmware oficial) es solo presentación GAP/SDP. El analizador del handshake no busca la cadena `ESP32`. La conexión usa la **dirección** remota más el UUID SPP o los atributos NUS.

1.1. Acoplamiento de modos

El transporte y la carga del firmware deben coincidir con el `BluetoothConnectionMode` almacenado:

Modo en la aplicación	Transporte	proto	Carga tras ACK
Classic Simple	SPP	`simple`	Sección 7
Classic Binary	SPP	`binary`	Sección 8
BLE Binary	NUS	`binary`	Sección 8

Simple | Binary | Simple heredado

BLE es solo Binary en la aplicación actual. Un perfil GATT serie propio (UUID típicos de HM-10) falla en el descubrimiento: el cliente solo busca Nordic UART.

La discrepancia debe señalarse con `NAK` (`expected` = capacidad del dispositivo, `actual` = valor de `CONNECT`). Ignorarla produce `HandshakeFailure.Timeout` en el teléfono.

2. Transportes

2.1. Classic SPP (RFCOMM)

Serial Port Profile, UUID `00001101-0000-1000-8000-00805F9B34FB`.

El cliente Android intenta, en este orden: `createRfcommSocketToServiceRecord` seguro, `createInsecureRfcommSocketToServiceRecord` inseguro y luego un socket RFCOMM por reflexión en el canal 1. El tercer intento existe porque algunos stacks (incluido `BluetoothSerial` del ESP32 con un `WifiNetworkSpecifier` SoftAP activo) fallan la búsqueda SDP del UUID.

HC-05 / HC-06 exponen este UUID como puente UART transparente. El MCU solo implementa el protocolo de bytes en ese UART; el baud es ajuste del módulo (habitualmente 9600 o 115200), no lo negocia el teléfono.

2.2. BLE Nordic UART Service

Atributo en el periférico	UUID	Rol
Servicio primario	`6E400001-B5A3-F393-E0A9-E50E24DCCA9E`	Servicio
RX (Write / Write Without Response)	`6E400002-...CCA9E`	Teléfono → dispositivo
TX (Notify)	`6E400003-...CCA9E`	Dispositivo → teléfono
CCCD en TX	`00002902-0000-1000-8000-00805F9B34FB`	Activar notify

Servicio · El teléfono escribe · El dispositivo notifica · Config. cliente

El cliente escribe octetos de protocolo en RX y recompone las notificaciones TX en el mismo flujo que SPP. Cada notify/write debe ser $\leq$ MTU-3. El MTU ATT por defecto (23) implica trozos de 20 bytes; pida un MTU mayor si envía tramas Binary de una sola vez. El handshake sigue siendo ASCII delimitado por newline en este flujo.

2.3. TCP

La misma sesión y carga que Classic, en una conexión TCP. El firmware SoftAP oficial enlaza `192.168.4.1:3333`. Host y puerto no forman parte del protocolo de aplicación; son parámetros de despliegue. No invente un segundo handshake para TCP.

3. Codificación de texto

Handshake y mensajes Simple. El color indica la función de cada token:

APP	TYPE	key1	value1	key2	value2
app	tipo	clave	valor	clave	valor

```
APP:TYPE,key1,value1,key2,value2,...
```

App Tipo Clave Valor

Terminados en `LF` (`0x0A`). Un `CR` previo se elimina. `APP` en este producto es `RC`. Las claves respetan mayúsculas tal como las envía el teléfono (`proto`, `lx`, ...).

Booleanos: `1/0`, `true/false`, `on/off` o cualquier entero distinto de cero.

El ensamblador de entrada trata un `0xCC` inicial como inicio de trama Binary de telemetría y el ASCII imprimible como texto. Tras el ACK en modo Binary, `0xAA`/`0xBB` son tramas de control hacia el dispositivo.

4. Handshake de sesión

Obligatorio en cada conexión nueva SPP, NUS o TCP. No se usa para la cámara HTTP.

4.1. Cliente → dispositivo

```
RC:CONNECT,proto,simple
RC:CONNECT,proto,binary
RC:CONNECT,proto,wifi
```

App Tipo Clave Valor

`wifi` se conserva para SoftAP CAM de una sola placa heredado. El firmware Simple nuevo debe aceptar `simple` y `wifi` y emitir el mismo ACK.

El teléfono espera 2500 ms una línea con prefijo `RC` y tipo `ACK` o `NAK`.

4.2. Dispositivo → cliente

```
RC:ACK,app,RC
```

Las claves extra del ACK se ignoran al establecer la sesión.

```
RC:NAK,reason,proto_mismatch,expected,simple,actual,binary
RC:NAK,reason,app_mismatch,expected,RC,actual,XX
```

`reason` debe ser `proto_mismatch` o `app_mismatch` para los diálogos estructurados de Android. Solo `RC:ACK,app,RC` abre la sesión. Un ACK posterior no es éxito de CONNECT. Los sketches de serie no envían un ACK de centro de dirección.

4.3. Estado recomendado en el dispositivo

1. Enlace arriba → `session_ok = false`, salidas en fail-safe.
2. Línea `CONNECT` completa → comparar `proto` (y, si se desea, el prefijo de app) → ACK o NAK.
3. Tras ACK, habilitar `CTRL/AA 55` y TX de telemetría.
4. Al caer el enlace, resetear `session_ok` y las salidas.

5. Implementación de referencia: SPP Simple (ESP32 `BluetoothSerial`)

Bluetooth Classic en piezas clase ESP32-WROOM. Ajuste de la aplicación: Classic Simple. El nombre GAP es arbitrario.

```
#include "BluetoothSerial.h"

#define SERIAL_BAUD         115200

BluetoothSerial SerialBT;

const char* BT_NAME = "TeleCon-RC";
const char* MY_PROTO = "simple";

bool sessionOk = false;
String lineBuf;

int lx = 0, ly = 0, rx = 0, ry = 0;
int lk = 512, rk = 512;
uint8_t sw = 0;

String valueOf(const String& line, const char* key) {
  String token = String(key) + ",";
  int i = line.indexOf(token);
  if (i < 0) return "";
  i += token.length();
  int j = line.indexOf(',', i);
  if (j < 0) j = line.length();
  return line.substring(i, j);
}

void sendLine(const String& s) {
  SerialBT.print(s);
  SerialBT.print('\n');
}

void applyOutputs() {
```

```
  // Mapear palancas ya condicionadas (-100..100) a actuadores.
}

void handleConnect(const String& line) {
  String proto = valueOf(line, "proto");
  if (proto != MY_PROTO &&
      !(String(MY_PROTO) == "simple" && proto == "wifi")) {
    sendLine(String("RC:NAK,reason,proto_mismatch,expected,") +
             MY_PROTO + ",actual," + proto);
    sessionOk = false;
    return;
  }
  sendLine("RC:ACK,app,RC");
  sessionOk = true;
}

void handleCtrl(const String& line) {
  lx = valueOf(line, "lx").toInt();
  ly = valueOf(line, "ly").toInt();
  rx = valueOf(line, "rx").toInt();
  ry = valueOf(line, "ry").toInt();
  lk = valueOf(line, "lk").toInt();
  rk = valueOf(line, "rk").toInt();
  sw = (uint8_t)strtol(valueOf(line, "sw").c_str(), nullptr, 16);
  applyOutputs();
}

void handleLine(String line) {
  line.trim();
  if (line.length() == 0) return;
  if (line.startsWith("RC:CONNECT")) {
    handleConnect(line);
    return;
  }
  if (!sessionOk) return;
  if (line.startsWith("RC:CTRL")) handleCtrl(line);
  else if (line.startsWith("RC:BTN")) {
    (void)valueOf(line, "id").toInt();
  } else if (line.startsWith("RC:SET")) {
    // Los sketches de serie ignoran RC:SET. El trim queda en el telfono
    .
  }
}

void setup() {
  Serial.begin(SERIAL_BAUD);
  SerialBT.begin(BT_NAME);
}
```

```
void loop() {
  if (!SerialBT.hasClient()) {
    if (sessionOk) {
      sessionOk = false;
      // fail-safe
    }
    lineBuf = "";
  }
  while (SerialBT.available()) {
    char c = (char)SerialBT.read();
    if (c == '\n') {
      handleLine(lineBuf);
      lineBuf = "";
    } else if (c != '\r') {
      if (lineBuf.length() < 200) lineBuf += c;
      else lineBuf = "";
    }
  }
}
```

Telemetría tras el ACK (DATA típico 2 Hz, PLOT 10–20 Hz):

```
void sendTelemetry(int leftPanel, int batt, int analog) {
  sendLine("RC:DATA,left," + String(leftPanel) +
           ",batt," + String(batt) + ",analog," + String(analog));
}

void sendPlot(uint8_t v0, uint8_t v1, uint8_t v2, uint8_t v3,
              uint8_t v4, uint8_t v5, uint8_t v6, uint8_t v7) {
  sendLine("RC:PLOT,v0," + String(v0) + ",v1," + String(v1) +
           ",v2," + String(v2) + ",v3," + String(v3) +
           ",v4," + String(v4) + ",v5," + String(v5) +
           ",v6," + String(v6) + ",v7," + String(v7));
}
```

6. Implementación de referencia: SPP Simple por UART HC-05

El módulo termina SPP; el MCU ve un UART. La misma gramática `CONNECT`/`CTRL`. Iguale el baud del módulo (a menudo 9600). Cruce TX/RX; adapte niveles si hace falta.

```
#include <Arduino.h>

#define SERIAL_BAUD         115200

HardwareSerial BT(1);
const int BT_RX_PIN = 16;
const int BT_TX_PIN = 17;
```

```
const uint32_t BT_BAUD = 9600;
const char* MY_PROTO = "simple";

bool sessionOk = false;
String lineBuf;

String valueOf(const String& line, const char* key) {
  String token = String(key) + ",";
  int i = line.indexOf(token);
  if (i < 0) return "";
  i += token.length();
  int j = line.indexOf(',', i);
  if (j < 0) j = line.length();
  return line.substring(i, j);
}

void sendLine(const String& s) {
  BT.print(s);
  BT.print('\n');
}

void handleConnect(const String& line) {
  String proto = valueOf(line, "proto");
  if (proto != MY_PROTO) {
    sendLine(String("RC:NAK,reason,proto_mismatch,expected,") +
             MY_PROTO + ",actual," + proto);
    sessionOk = false;
    return;
  }
  sendLine("RC:ACK,app,RC");
  sessionOk = true;
}

void handleLine(String line) {
  line.trim();
  if (line.startsWith("RC:CONNECT")) { handleConnect(line); return; }
  if (!sessionOk) return;
  if (line.startsWith("RC:CTRL")) {
    int ly = valueOf(line, "ly").toInt();
    int rxStick = valueOf(line, "rx").toInt();
    (void)ly;
    (void)rxStick;
  }
}

void setup() {
  Serial.begin(SERIAL_BAUD);
  BT.begin(BT_BAUD, SERIAL_8N1, BT_RX_PIN, BT_TX_PIN);
}
```

```
void loop() {
  while (BT.available()) {
    char c = (char)BT.read();
    if (c == '\n') { handleLine(lineBuf); lineBuf = ""; }
    else if (c != '\r') {
      if (lineBuf.length() < 200) lineBuf += c;
      else lineBuf = "";
    }
  }
}
```

7. Carga Simple

7.1. Teléfono → dispositivo

RC:CTRL Se emite al cambiar la UI (no es un sondeo fijo en el teléfono).

```
RC:CTRL,lx,-50,ly,100,rx,0,ry,0,lk,512,rk,256,sw,03
```

Clave	Dominio	Significado
lx, ly, rx, ry	−100 . . . 100	Palancas
lk, rk	0 . . . 1023	Potenciómetros
sw	hex 8 bits, dos dígitos	S1 = bit0 . . . S8 = bit7

Palancas Potenciómetros Interruptores

Dual-rate, expo, inverso, zona muerta y el trim de dirección se aplican en el teléfono antes del TX. El firmware no debe volver a aplicar esas curvas. Los sketches de serie no guardan un centro de servo: `rx` = 0 es un pulso de 1500 µs. El trim del teléfono (`rightTrimX`) ya va dentro de `rx`. Un sketch de vehículo posterior puede guardar un centro por su cuenta si un segundo transmisor debe compartir esa línea recta.

RC:BTN `RC:BTN,id,1` — id 1–4, el cluster central. El teléfono no envía el id 16.

RC:SET Los sketches de serie ignoran `RC:SET`. No persista un centro de dirección a partir de esta línea.

7.2. Dispositivo → teléfono

La telemetría es opcional. Envíela solo tras el ACK, en el mismo flujo de texto que `CTRL`. Hay dos tipos de línea: `DATA` para paneles e indicadores lentos, y `PLOT` para un bus analógico rápido. Si el firmware no tiene nada que mostrar, omita el tipo; el teléfono conserva los últimos valores.

`RC:DATA` — paneles, indicadores, LED (típico 2 Hz) Cada línea es una instantánea de campos enteros del HUD. Las claves pueden ir en cualquier orden. Si falta una clave, el teléfono mantiene el valor anterior.

```
RC:DATA,left,1234,right,5678,analog,42,batt,88,led,0F
```

Clave	Dominio	Significado	HUD típico
`left`	entero	Panel numérico izquierdo	RC Vehicle Pro: velocidad×10
`right`	entero	Panel numérico derecho	Segundo contador / sin uso
`analog`	0 . . . 255	Indicador analógico	Temperatura del motor
`batt`	0 . . . 255	Indicador de batería	Basta con carga 0 . . . 100
`led`	hex 8 bits, dos dígitos	Bits de LED	Lámparas de estado

Paneles Indicadores LED

El encendido y el color de los paneles se configuran en Android, no en el cable. Las claves heredadas `lo`, `ro`, `lg`, `rg`, `lt`, `rt`, `at`, `bt` se ignoran para la apariencia.

El teléfono puede reasignar estas claves (y los canales PLOT) a gráficas, radar o indicadores en los ajustes RC. El firmware debe seguir enviando las mismas claves. No emita `RC:RADAR` ni paquetes de mapa de canales.

`RC:PLOT` — muestras analógicas (típico 10–30 Hz) Cada línea es una muestra de tiempo en el bus numerado `v0`. . . `v7` (= CH1. . . CH8). Mapee sensores a 0 . . . 255 (0 = abajo de la gráfica, 255 = arriba). Los sketches Bluetooth de serie envían las ocho claves cada 50 ms aproximadamente. Una línea de cuatro muestras sigue siendo válida; si omite `v4`. . . `v7`, el teléfono los deja en 0. La aplicación guarda ocho canales y los últimos 100 puntos de cada uno; una novena muestra se descarta.

```
RC:PLOT,v0,128,v1,200,v2,64,v3,180
RC:PLOT,v0,128,v1,200,v2,64,v3,180,v4,0,v5,0,v6,0,v7,0
```

Clave	Canal	Rango	Notas
`v0`	CH1	0 . . . 255	Primera traza por defecto
`v1`	CH2	0 . . . 255	Segunda traza / radar por defecto
`v2`	CH3	0 . . . 255	Tercera traza por defecto
`v3`	CH4	0 . . . 255	Cuarta traza por defecto
`v4`. . . `v7`	CH5. . . CH8	0 . . . 255	El mismo bus; asígnelos en el mapa de canales

CH1–CH2 CH3–CH4 CH5–CH8

Control Panel muestra cuatro widgets de gráfica (por defecto CH1–CH4). Los colores de traza están fijos por índice de canal. Las etiquetas y el mapa de canales viven en los ajustes RC de Android — no envíe `RC:PLOTCFG`. Esos mismos ocho bytes alimentan el radar, los indicadores y las insignias del HUD cuando el usuario elige ese canal.

Tabla de canales Bluetooth de serie. Classic Simple, Classic Binary y BLE Binary rellenan los ocho bytes desde `kTelemetryChannels[]` (fila 0 = CH1). Classic Simple guarda la tabla en el `.ino`; los sketches binarios la guardan en `TelemetryChannels.h`. Una fila es `{ kind, vehicle, pin, ledcChannel, dutyMin, dutyMax }`. Solo se leen los campos del `kind`.

kind	Byte	Campos usados
`TEL_VEHICLE`	Señal interna nombrada por `vehicle`	`vehicle`
`TEL_ADC`	`analogRead`, $0 \ldots 4095 \mapsto 0 \ldots 255$	`pin`
`TEL_GPIO`	`digitalRead` $\mapsto 0$ o 255	`pin`
`TEL_PWM`	Ciclo LEDC, `dutyMin`$\mapsto 0$, `dutyMax`$\mapsto 255$	`pin`, canal, ventana

En una fila de pin se ignora el nombre `vehicle`. En una fila `TEL_VEHICLE` se ignoran el pin y la ventana de duty. Los nombres internos son temperatura, acelerador (ly), batería, dirección (rx), luces, ambos mandos y ry. Esas magnitudes no indican el sentido. `TEL_GPIO` en un pin de salida informa el nivel escrito por última vez. `TEL_PWM` lee un pin ya configurado con LEDC. Un PWM de motor de 8 bits usa duty $0 \ldots 255$. Un servo de 12 bits en 1000–2000 µs usa duty $204 \ldots 409$ (centro ≈ 128). El core 3.x lee el GPIO; el core 2.x lee `ledcChannel`.

Valores de serie en Classic Simple: temperatura de demostración, magnitud del acelerador, batería de demostración, magnitud del stick de dirección, interruptor de luces, ambos mandos y el PWM del servo de dirección. En Classic Binary y BLE Binary, con la E/S del vehículo activa: temperatura, magnitud del acelerador, batería escalada a $0 \ldots 255$, magnitud del stick de dirección, PWM de tracción izquierda (GPIO 25), PWM de tracción derecha (GPIO 27), GPIO de luces 15 y PWM del servo de paneo (GPIO 12, la misma ventana $204 \ldots 409$).

Los dos ejes de los dos sticks caben en cuatro de las ocho filas. Mapee un eje con signo y centro 128: `map(axis, -100, 100, 0, 255)`. El periodo del PLOT sigue en 50 ms; ocupar más de los ocho bytes no añade un paquete. Una línea Simple ocupa unos 80 bytes. El `CC 33` binario con ocho muestras ocupa 14 bytes.

`loop()` típico: PLOT cada 30–100 ms desde ADC o sensores filtrados; DATA cada 500 ms para paneles y batería. Condicione ambos a `session_ok`.

Inserte esto en el sketch de la sección 5. `sendLine`, `sessionOk`, `ly`, `rx`, `lk`, `rk` y `sw` ya están ahí.

```
int toPlotByte(int value, int minVal, int maxVal) {
  if (maxVal <= minVal) return 0;
  return constrain(map(value, minVal, maxVal, 0, 255), 0, 255);
}

void sendData(int left, int right, int analog, int batt, uint8_t led) {
  char line[96];
```

```
  snprintf(line, sizeof(line),
           "RC:DATA,left,%d,right,%d,analog,%d,batt,%d,led,%02X",
           left, right, analog, batt, led);
  sendLine(line);
}

void sendPlot(uint8_t v0, uint8_t v1, uint8_t v2, uint8_t v3,
              uint8_t v4, uint8_t v5, uint8_t v6, uint8_t v7) {
  char line[128];
  snprintf(line, sizeof(line),
           "RC:PLOT,v0,%u,v1,%u,v2,%u,v3,%u,v4,%u,v5,%u,v6,%u,v7,%u",
           v0, v1, v2, v3, v4, v5, v6, v7);
  sendLine(line);
}

unsigned long lastDataMs = 0;
unsigned long lastPlotMs = 0;

void loop() {
  // leer SerialBT y handleLine() como en la seccion 5
  if (!sessionOk) return;

  unsigned long now = millis();
  if (now - lastDataMs >= 500) {
    lastDataMs = now;
    int speedX10 = abs(ly) * 25 / 10;  // HUD de RC Vehicle Pro
    sendData(speedX10, 0, 90, 76, sw);
  }
  if (now - lastPlotMs >= 50) {
    lastPlotMs = now;
    sendPlot(toPlotByte(analogRead(34), 0, 4095),
             toPlotByte(abs(ly), 0, 100),
             toPlotByte(76, 0, 100),
             toPlotByte(abs(rx), 0, 100),
             (sw & 0x01) ? 255 : 0,
             toPlotByte(lk, 0, 1023),
             toPlotByte(rk, 0, 1023),
             128);
  }
}
```

8. Carga Binary

Se usa tras `RC:CONNECT,proto,binary` y ACK. El handshake sigue siendo ASCII.

8.1. Teléfono → dispositivo — `AA 55` (18 bytes)

Canales de 12 bits little-endian. El color indica la función de cada campo:

`AA 55`	`Lx`	`Ly`	`Rx`	`Ry`	`Lk`	`Rk`	`SW`	$\sum$	`CR LF`
0–1	2–3	4–5	6–7	8–9	10–11	12–13	14	15	16–17

Despl.	Tam.	Contenido	Rol
0–1	2	Sincronía `0xAA 0x55`	Sincronía
2–3	2	Lx, 12 bit LE	Canal
4–5	2	Ly, 12 bit LE	Canal
6–7	2	Rx, 12 bit LE	Canal
8–9	2	Ry, 12 bit LE	Canal
10–11	2	Potenciómetro izquierdo 0 . . . 1023, 12 bit LE	Potenciómetro
12–13	2	Potenciómetro derecho 0 . . . 1023, 12 bit LE	Potenciómetro
14	1	Bits de interruptores (S1 = bit0 . . . S8 = bit7)	Interruptores
15	1	$\sum$ bytes 2 . . . 14 (8 bits bajos)	Checksum
16–17	2	Terminador `0x0D 0x0A`	Terminador

Sincronía | Canales | Potenciómetros | Interruptores | Checksum | Terminador

Mapeo de palanca: porcentaje $p \in [-100, 100] \mapsto \lfloor (p + 100) \times 4094/200 \rfloor$ con centro $0 \mapsto 2047$ (unsigned 0 . . . 4094).

8.2. Teléfono → dispositivo — `BB 66` (4 bytes)

Los id coinciden con `RC:BTN` Simple.

Despl.	Tam.	Contenido	Rol
0–1	2	Sincronía `0xBB 0x66`	Sincronía
2	1	Id de botón (1–4, cluster central)	Botón
3	1	El mismo id repetido	Botón

Sincronía | Botón

8.3. Dispositivo → teléfono

`CC 11` (8 bytes)

Despl.	Tam.	Contenido	Rol
0–1	2	Sincronía `0xCC 0x11`	Sincronía
2–3	2	left, int16 LE	Telemetría
4–5	2	right, int16 LE	Telemetría
6	1	Flags reservados	Telemetría
7	1	$\sum$ bytes 2 . . . 6 (8 bits bajos)	Checksum

`CC 22` (6 bytes)

Despl.	Tam.	Contenido	Rol
0–1	2	Sincronía `0xCC 0x22`	Sincronía
2	1	analog, $u8$	Telemetría
3	1	battery, $u8$	Telemetría
4	1	LED, $u8$	Telemetría
5	1	$\sum$ bytes 2 . . . 4 (8 bits bajos)	Checksum

`CC 33` (variable) Los sketches Bluetooth de serie envían `count = 8` (trama de 14 bytes). `count = 4` sigue siendo válido; el teléfono deja entonces CH5–CH8 en 0. No envíe `CC 44`. Las ocho muestras son el mismo bus CH1–CH8 que `RC:PLOT` de Simple, rellenadas desde `kTelemetryChannels[]` en `TelemetryChannels.h`.

Despl.	Tam.	Contenido	Rol
0–1	2	Sincronía `0xCC 0x33`	Sincronía
2–3	2	`length` uint16 LE ($= 1 +$ *count*)	Longitud
4	1	`count` (4 u 8)	Cuenta
5–4+c	c	c muestras $u8$ (CH1 . . . CHc)	Muestras
5+c	1	$\sum$ desde offset 2 hasta la última muestra	Checksum

$c =$ `count`. El checksum cubre desde el desplazamiento 2 hasta la última muestra.

9. Fuera de alcance

- HTTP MJPEG y `/camconfig`.
- Asignación oficial de SSID SoftAP (despliegue, no tramas).
- Perfiles BLE serie que no sean NUS.
- Handshake opcional; el cliente no confirmará la sesión.

10. Lista de implementación

1. Enlazar SPP, NUS o TCP como flujo de octetos.
2. Acumular hasta `LF`; analizar `RC:CONNECT`.
3. Responder `RC:ACK,app,RC` o un `NAK` estructurado.
4. Condicionar control/telemetría a `session_ok`; fail-safe al desconectar.
5. Simple: `CTRL`/`BTN`/`SET` y, si aplica, `DATA`/`PLOT`.
6. Binary: `AA 55`/`BB 66` y, si aplica, `CC 11`/`CC 22`/`CC 33`.
7. Documentar para qué modo de la aplicación se compiló el binario.